

Prompt-State Machines: Applying Interface-Led Architecture to Structure LLM Reasoning

Takuya Sogawa

Prompt-State Machines

Applying Interface-Led Architecture to Structure LLM Reasoning

Version: v1.0.0

Type: Technical Note / ILA Supplement 2

Author: Takuya Sogawa

ORCID: <https://orcid.org/0009-0009-7499-2595>

DOI: 10.5281/zenodo.20323512

License: CC BY 4.0

Abstract

This technical note proposes **Prompt-State Machines** as a prompt-level application of Interface-Led Architecture (ILA) and its Provider-Observer-Operator abstraction discipline.

Large Language Models (LLMs) are stochastic generators. Their internal inference process cannot be made fully deterministic by prompt design alone. However, a structured prompt can guide and constrain the model's output process by presenting reasoning as an explicit sequence of roles, states, transitions, and failure paths. In this sense, a prompt can behave as a lightweight pseudo state machine: not as a kernel-enforced deterministic automaton, but as a prompt-level governance protocol for structured reasoning.

This paper applies the Provider-Observer-Operator model to LLM reasoning. A **Provider** supplies assumptions, available knowledge, constraints, and input materials. An **Observer** inspects ambiguity, missing information, contradictions, risk, and unsafe assumptions. An **Operator** performs transformation, reasoning, synthesis, or answer generation only after the Provider and Observer stages have completed.

The paper further argues that Markdown, and more specifically ROM-like Markdown, is a practical state-boundary representation for LLMs because it is simultaneously human-readable, model-readable, section-oriented, and easy to review. By embedding `Clarify` and `FailClosed` transitions into the prompt, a Prompt-State Machine prevents ambiguous or unsafe requests from being silently promoted into final answers. The paper also includes a practical Skill-Set Provider pattern for declaring available reasoning capabilities inside the Provider state.

The contribution of this paper is to define a lightweight governance model that transfers ILA's role-based abstraction discipline into prompt space, allowing LLM reasoning to be structured as state boundaries, role responsibilities, and failure transitions.

Keywords

Interface-Led Architecture; ILA; Prompt-State Machine; Provider; Observer; Operator; LLM Reasoning; Prompt Engineering; Markdown; ROM; Fail-Closed; Clarify; Governed Reasoning; AIKernel

1. Introduction

Prompt engineering is often described as the craft of writing better instructions. That description is useful but incomplete. When an LLM is used as part of an autonomous agent, tool-using assistant, or AI operating-system layer, a prompt is not merely an instruction. It becomes a boundary object that defines reasoning order, responsibility separation, failure behavior, and the conditions under which the model should refuse to proceed.

Unstructured prompts frequently collapse several responsibilities into a single request. The model is asked to infer missing assumptions, observe contradictions, evaluate safety, choose a method, and generate a final answer at once. Because LLMs are trained to continue text, they often proceed even when the input is underspecified, contradictory, or unsafe.

This paper argues that the problem is not only model capability. It is also the absence of state boundaries in the prompt itself.

Interface-Led Architecture (ILA) provides a useful abstraction discipline for this problem. In ILA, interface-bearing components are classified into primitive roles: Provider, Observer, and Operator. Higher-level structures are Units composed of those roles and connected through Pipelines under Core Contracts.

This paper transfers that discipline into prompt space. It defines a **Prompt-State Machine** as a structured prompt that separates the model's task into Provider, Observer, and Operator states, with explicit Clarify and FailClosed transitions. In addition to the minimal form, the paper presents a Skill-Set Provider pattern showing how available reasoning capabilities can be declared before they are observed and used.

2. Position in the ILA / AIKernel Series

This paper is a supplement to the ILA paper and the Provider-Observer-Operator abstraction discipline.

The conceptual lineage is:

Interface-Led Architecture:

Interface-first software methodology.

Provider-Observer-Operator:

Role-based abstraction discipline for extracting interfaces.

Prompt-State Machines:

Application of that abstraction discipline to LLM prompt space.

This paper does not extend the general ILA theory itself. It also does not import AIKernel-specific specializations such as Semantic IR, Composite Distance, Governed Circuits, or ReplayLog directly into prompt space.

Instead, this paper treats Prompt-State Machines as an **application model**: a way of applying ILA's role-based abstraction discipline to the structure of prompts. It is therefore situated below the general ILA theory and beside AIKernel's runtime architecture.

The distinction is important:

ILA general theory:
Role / Unit / Pipeline / Core Contract.

AIKernel specialization:
Semantic IR / Composite Distance / ReplayLog / Governed Circuit.

Prompt-State Machine:
Prompt-level role sequencing and failure transitions.

3. Problem Statement

A typical prompt asks the model to produce an answer directly:

Solve this problem.

Such a prompt leaves the model to perform several tasks implicitly:

- identify assumptions;
- infer missing context;
- detect contradictions;
- classify ambiguity;
- choose a solution method;
- decide whether to ask a question;
- decide whether answering would be unsafe;
- generate the final response.

These responsibilities are not the same. They correspond to different architectural roles.

A model that answers before observing missing information is operating without an Observer boundary. A model that generates a solution before enumerating assumptions is operating without a Provider boundary. A model that proceeds despite unresolved ambiguity is operating without a failure transition.

Prompt-State Machines address this by structuring the prompt as a sequence of states:

Provider -> Observer -> Operator

The key rule is that the Operator stage must not be entered if the Observer stage detects missing required information, contradiction, unsafe execution intent, or non-governable ambiguity. In such cases, the prompt should transition to Clarify or FailClosed instead.

4. Prompt-State Machine Model

A Prompt-State Machine (PSM) is a structured prompt that defines states, role responsibilities, transition order, and failure paths.

This paper models a PSM as:

$PSM = (M, S, T, R, F)$

where:

- M is metadata, typically expressed in YAML front matter;
- S is the set of states;
- T is the transition relation;
- R is the responsibility assigned to each state;
- F is the set of failure transitions, such as Clarify and FailClosed.

In the minimal model:

`S = { Provider, Observer, Operator, Clarify, FailClosed }`

The normal transition path is:

`Provider -> Observer -> Operator -> Answer`

The guarded transition paths are:

`Provider -> Observer -> Clarify`

`Provider -> Observer -> FailClosed`

This is not a deterministic state machine in the formal automata-theoretic sense. The LLM remains probabilistic. The structure is instead a prompt-level pseudo state machine that constrains reasoning order and makes failure paths explicit.

5. Provider State

The Provider state supplies the materials required for reasoning. It identifies assumptions, available information, constraints, known facts, and capability boundaries.

A Provider does not generate the final answer. It also does not evaluate whether the input is ambiguous or unsafe. Its responsibility is to make the available material explicit.

Typical Provider instructions are:

`# Provider`

- List the assumptions needed to solve the task.
- Organize the information explicitly given by the user.
- Identify available constraints, data, and permitted resources.
- If required information is missing, record it as Unknown.
- Do not guess missing required information.

The Provider state is valuable because it prevents the model from hiding assumptions inside the final answer. It externalizes what the model believes it has been given.

6. Observer State

The Observer state inspects the materials supplied by the Provider. It detects ambiguity, missing required information, contradiction, risky assumptions, unsafe execution intent, and unsupported claims.

An Observer does not produce the final answer. It also should not mutate the reasoning state into a solution. It provides evidence used to decide whether the Operator stage may proceed.

Typical Observer instructions are:

`# Observer`

- Inspect the input for contradictions.
- Detect ambiguous terms or undefined conditions.
- List missing information required for a safe answer.
- Separate assumptions that may be safely inferred from assumptions that must not be guessed.
- If clarification is required, generate the clarification question.
- If the request is unsafe or non-governable, transition to FailClosed.

This stage is the most important safety boundary in the Prompt-State Machine. It prevents the model from treating all prompts as answerable.

7. Operator State

The Operator state performs transformation, reasoning, synthesis, planning, or final answer generation. It acts only after the Provider has supplied the materials and the Observer has determined that the task may proceed.

Typical Operator instructions are:

Operator

- Construct the solution using the Provider output and the Observer findings.
- Do not invent missing required information.
- If the Observer requested Clarify, do not generate a final answer.
- Answer only within the safely supported scope.
- If safe completion is impossible, return Clarify or FailClosed.

The Operator is the only stage that should produce the final answer. This separation makes the model's reasoning path more auditable and easier to review.

8. Clarify and FailClosed Transitions

A Prompt-State Machine does not connect every input to an answer.

The Clarify transition asks the user for missing information. The FailClosed transition refuses to proceed when the prompt cannot be safely or meaningfully completed.

The following transition rules are typical:

Observer detects missing required information -> Clarify

Observer detects contradiction -> Clarify

Observer detects unsafe execution request -> FailClosed

Observer cannot verify required assumptions -> Clarify or FailClosed

This is the prompt-level analogue of fail-closed design. Instead of guessing under uncertainty, the model is instructed to stop, ask, or refuse.

9. Markdown / ROM as State Boundary Representation

Markdown is a practical state-boundary representation for Prompt-State Machines.

Its advantages are architectural, not merely stylistic:

- headings separate states;
- YAML front matter defines metadata;
- lists express responsibilities;
- code blocks define input and output formats;
- humans can inspect the structure easily;
- LLMs are generally able to follow the structure reliably.

ROM-like Markdown strengthens this pattern by treating a prompt not as an unstructured string, but as a governed document containing role sections, relation hints, constraints, and failure behavior.

This paper does not claim that Markdown is the only valid state-boundary representation. XML, JSON, YAML, DSLs, and schema-enforced tool calls can also express state. Markdown is emphasized because it provides a pragmatic balance between human readability and model readability.

10. Minimal Example

The following is a minimal Prompt-State Machine:

```
---
task: "problem solving"
```

```
pipeline:
```

- provider
- observer
- operator

```
fail_closed: true
```

```
clarify_when:
```

- missing_required_information
- contradiction_detected
- unsafe_execution_request

```
---
```

Provider

- List the assumptions required to solve the problem.
- Organize the information explicitly provided by the user.
- If information is missing, mark it as Unknown.
- Do not guess missing required information.

Observer

- Inspect the input for contradictions, ambiguity, and omissions.
- Separate safe assumptions from assumptions that must not be guessed.
- If clarification is required, generate the question.
- Do not proceed to Operator if clarification is required.

Operator

- Construct the solution using Provider and Observer outputs.
- Do not generate a final answer if Observer requested Clarify.
- Generate the final answer only within the safely supported scope.

The same idea can be represented in Japanese:

```
---
task: " 問題解決"
```

```
pipeline:
```

- provider
- observer
- operator

```
fail_closed: true
```

```
---
```

Provider

- 問題を解くために必要な前提知識を列挙せよ
- 与えられた入力情報を整理せよ
- 不足している情報があれば Unknown として明示せよ

Observer

- 入力の矛盾・曖昧さ・不足を観測せよ
- 推測してよい情報と推測してはいけない情報を分離せよ
- 必要なら質問を生成し、Operator へ進むな

Operator

- Provider の前提と Observer の観測結果に基づいて解法を構築せよ
- Observer が Clarify を要求している場合、最終回答を生成してはならない
- 安全に回答可能な範囲でのみ最終回答を生成せよ

10.1 Skill-Set Provider Example

The minimal example above is intentionally generic. In practical use, the Provider state should often describe not only task inputs, but also the model's available skill set. A skill set is a prompt-level declaration of what kinds of reasoning, transformation, or domain handling the model is allowed to use for the current task.

A Skill-Set Provider is a specialization of the general Prompt-State Machine structure. In the general model, Provider supplies materials for reasoning. In this specialization, Provider also supplies the boundary of available capabilities: what skills may be used, what inputs each skill requires, what assumptions are forbidden, and what output contract must be respected. This preserves the general ILA role model while making the practical prompt boundary more explicit.

This is important because an LLM tends to infer its own capabilities from the request. If the prompt does not explicitly define the available skills, the model may silently assume abilities that are outside the desired task boundary. A Skill-Set Provider makes the available reasoning resources explicit before the Observer and Operator stages proceed.

A practical Skill-Set Provider may include the following fields:

Field	Meaning
skill_id	Stable identifier for the skill.
purpose	What the skill is allowed to support.
allowed_operations	Operations the Operator may use when this skill is active.
required_inputs	Inputs that must be present before the skill can be used.
forbidden_assumptions	Assumptions that must not be invented.
output_contract	Expected structure of the skill output.
fallback	Transition to use when required information is missing.

The following example defines a prompt-level skill set for technical problem solving:

```
---
task: "technical problem solving"
pipeline:
  - provider
  - observer
  - operator
fail_closed: true
skill_set:
  - skill_id: requirements_extraction
    purpose: "Extract explicit requirements, constraints, and unknowns."
```

```

allowed_operations:
  - list_requirements
  - identify_constraints
  - mark_unknowns
required_inputs:
  - user_request
forbidden_assumptions:
  - inventing unstated requirements
  - assuming hidden business rules
output_contract: "requirements, constraints, unknowns"
fallback: clarify

- skill_id: solution_design
  purpose: "Construct a solution only from validated Provider and Observer outputs."
  allowed_operations:
    - propose_solution
    - compare_options
    - explain_tradeoffs
  required_inputs:
    - requirements
    - observer_findings
  forbidden_assumptions:
    - using missing information as if it were known
    - bypassing Observer warnings
  output_contract: "solution, rationale, limitations"
  fallback: fail_closed
---
```

Provider

- Read the ``skill_set`` section.
- List the skills available for this task.
- For each skill, list its required inputs and forbidden assumptions.
- Mark unavailable skills if their required inputs are missing.

Observer

- Verify whether each required input is present.
- Detect whether any forbidden assumption would be needed.
- If a skill cannot be used safely, transition to Clarify or FailClosed.

Operator

- Use only skills that Provider marked as available and Observer validated.
- Do not use a skill if its required inputs are missing.
- Produce the answer according to the declared output contract.

This example illustrates how Provider can carry capability information without turning the prompt into a full runtime system. The skill set does not enforce execution like an AIKernel Core Contract. It makes the model's assumed reasoning resources visible, inspectable, and governable at the prompt level.

The design principle is simple: skills should be declared before they are used, observed before they are trusted, and operated only after their required inputs and boundaries are validated.

11. Relationship to SGP and Semantic Runtime Theory

Prompt-State Machines correspond structurally to the AIKernel SGP Pipeline and Semantic Runtime Theory, but they are not identical to them.

SGP separates Structure, Generation, and Polish. AIKernel Semantic Runtime Theory treats human intent as an observable semantic state that can be governed, replayed, and connected to execution. Prompt-State Machines provide a lighter prompt-level version of the same architectural intuition: reasoning is safer when responsibilities and transitions are explicit.

Prompt-State Machine	AIKernel Runtime / Semantic Runtime	Meaning of the correspondence
Provider	ROM / VFS / Context Provider	Supplies assumptions, knowledge, constraints, and materials required for reasoning.
Observer	Admissibility Gate / Trajectory Observer / Risk Observer	Observes ambiguity, missing information, candidate actions, risk, and unsafe assumptions.
Operator	Inference Operator / Transform Operator / Synthesis Operator	Performs reasoning, transformation, answer generation, or structured synthesis based on observed evidence.
Clarify	Clarify disposition / User confirmation path	Stops final-answer generation and asks for information required to proceed safely.
FailClosed	Deny / Abort / Safe fallback	Stops answer generation or execution when continuation is unsafe or non-governable.
Markdown / ROM-like Prompt	Structured Context / ROM boundary	Provides a state-boundary representation readable by both humans and LLMs.
Prompt-State Machine	Governed reasoning pattern	Provides prompt-level lightweight governance rather than runtime-level enforcement.

The correspondence can be summarized as:

Prompt-State Machine:
Prompt-level governed reasoning pattern.

AIKernel Governed Circuit:
Runtime-level governed execution topology.

A Prompt-State Machine is not a simplified implementation of a Governed Circuit. It is the prompt-space counterpart of governed execution architecture.

12. Relationship to Chain-of-Thought and ReAct

Chain-of-Thought prompting showed that eliciting intermediate reasoning steps can improve performance on complex reasoning tasks. ReAct showed that reasoning traces and action steps can be interleaved so that language models reason, act, observe, and update plans.

Prompt-State Machines are related to these approaches, but the objective is different.

The purpose is not merely to elicit longer reasoning traces or to interleave action with observation. The purpose is to impose architectural responsibility separation on the prompt itself:

- Provider supplies materials;
- Observer inspects whether the task may proceed;
- Operator acts only after observation;
- Clarify and FailClosed prevent unsafe continuation.

Thus, Prompt-State Machines should be understood as a governance-oriented prompt structure rather than a performance-oriented prompting trick.

13. Limitations and Non-Claims

This paper does not claim that prompt structure can fully control the internal inference process of an LLM. LLMs remain stochastic generators, and prompt-level structure is not equivalent to kernel-level enforcement.

This paper also does not claim that Markdown or ROM is the only valid prompt-state representation. Other structured formats may be appropriate in different environments.

Prompt-State Machines do not replace AIKernel Runtime Governance, Pre-Inference Admissibility Governance, Trajectory Governance, or Semantic Runtime Theory. They provide a prompt-level governance pattern that reflects the same design intuition under weaker enforcement conditions.

Finally, this paper does not claim that intermediate reasoning should always be exposed to end users. A Prompt-State Machine may be used internally to structure reasoning while presenting only the necessary final response, clarification, or refusal externally.

14. Conclusion

This paper defined Prompt-State Machines as a prompt-level application of Interface-Led Architecture and the Provider-Observer-Operator abstraction discipline.

An LLM is a stochastic generator, but a structured prompt can guide its reasoning process through explicit states and failure transitions. Provider supplies assumptions and materials. Observer detects ambiguity, contradiction, insufficiency, and risk. Operator transforms the observed state into a solution or response. Clarify and FailClosed prevent unsafe or underspecified inputs from being silently converted into final answers.

Markdown and ROM-like structures provide a practical state-boundary representation because they are simultaneously human-readable, model-readable, and easy to review.

Prompt-State Machines correspond structurally to AIKernel’s SGP Pipeline, Trajectory Governance, and Semantic Runtime Theory, while remaining prompt-level governance protocols rather than runtime-enforced circuits.

In this sense, ILA operates not only in implementation space but also in prompt space: it provides a method for organizing roles, boundaries, and transitions wherever reasoning must be made governable.

The structure proposed here also leaves room for future experimental evaluation across multiple models to assess the effectiveness and reproducibility of Prompt-State Machines as a prompt-level governance pattern.

References

1. Sogawa, Takuya. "Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model."Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
2. Sogawa, Takuya. "Provider-Observer-Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture."Zenodo, 2026. DOI: 10.5281/zenodo.20322690.
3. Sogawa, Takuya. "AIKernel Phase-2 Theory: Semantic Compilation Architecture."Zenodo, 2026. DOI: 10.5281/zenodo.20312092.
4. Harel, David. "Statecharts: A Visual Formalism for Complex Systems."Science of Computer Programming, vol. 8, no. 3, 1987, pp. 231-274. DOI: 10.1016/0167-6423(87)90035-9.
5. Wei, Jason, Xuezhong Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." arXiv:2201.11903, 2022.
6. Yao, Shunyu, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. "ReAct: Synergizing Reasoning and Acting in Language Models."International Conference on Learning Representations (ICLR), 2023.
7. CommonMark. "CommonMark: A Strongly Defined, Highly Compatible Specification of Markdown."CommonMark Specification. Available at: <https://commonmark.org/>.